

Inter-Agent Coordination Protocol for COA

Executive Summary

The protocol addresses cross-boundary collisions between parallel agents using three layers: (1) a BBA-first rule that converts most modifications into conflict-free additions, (2) a lightweight file-based claims mechanism for the remaining cases, and (3) a check script integrated into the existing gate infrastructure.

Key Insight: BBA Already Solves 80% of the Problem

Before designing coordination machinery, notice what COA already provides:

Defense Layer	What It Prevents
Hex module boundaries	Agents touching code outside their module
public-api.mjs barrel	Cross-module coupling to internals
BBA seams + feature-seams module	Modifications to active code paths
Atomic slices + fast commits	Long conflict windows
Structured headers	Misunderstanding file intent

The critical observation: **if every cross-boundary change is framed as an ADDITION (new export, new adapter, new seam) rather than a MODIFICATION of existing code, conflicts become structurally impossible**. Two agents can both add new exports to the same `public-api.mjs` --- git merges additions cleanly. Two agents cannot both modify the same existing function signature without a collision.

BBA-first rule: When Agent A needs to change module-beta's public API, it MUST first attempt to add a new export behind a BBA seam rather than modify an existing one. Only when modification is unavoidable (bug fix, rename, breaking change) does coordination become necessary.

This rule is the protocol's primary mechanism. Everything below handles the remaining ~20%.

1. Claims Mechanism

Location and Format

Claims live in `.claims/` at repo root. One JSON file per active claim.

```
.claims/  
  README.md  
  <claim-id>.json  
  <claim-id>.json
```

Claim File Schema

```
{  
  "id": "clm-a1b2c3",  
  "agent": "team-alpha / Igor / claude-session-xyz",
```

```

"slice": "TPL-123",
"created": "2026-04-03T10:00:00Z",
"expires": "2026-04-03T18:00:00Z",
"status": "active",
"targets": [
  {
    "path": "modules/auth/public-api.mjs",
    "module": "auth",
    "surface": "public-api",
    "action": "extend",
    "description": "Add OAuth2 PKCE token refresh export"
  }
],
"strategy": "bba-additive",
"dependsOn": [],
"notes": ""
}

```

Field definitions:

Field	Required	Description
id	yes	Short unique ID. Convention: c1m-<6-hex-chars>
agent	yes	Who filed this --- agent team ID, human name, or session ID
slice	yes	Backlog slice ID (links to docs/backlog/)
created	yes	ISO-8601 timestamp
expires	yes	ISO-8601 timestamp. Default: 8 hours from creation
status	yes	active / completed / abandoned
targets	yes	Array of files/surfaces this claim covers
targets[].path	yes	Relative file path
targets[].module	no	Module name (derived from path if absent)
targets[].surface	yes	One of: public-api, port, adapter, domain, shared-infra, config
targets[].action	yes	extend (additive) / modify (changing existing) / replace (rewrite)
targets[].description	yes	Why this file needs to change
strategy	yes	bba-additive (preferred) / modify-in-place / negotiate
dependsOn	no	Array of other claim IDs that must complete first
notes	no	Free-text for human/agent coordination

When Claims Are Required

Situation	Claim Needed?
-----------	---------------

Working entirely within your own module	No
Adding a new file to your own module	No
Adding a new export to another module's <code>public-api.mjs</code> behind a BBA seam	Advisory --- file a claim but proceed
Modifying an existing export in another module's <code>public-api.mjs</code>	Required
Modifying a shared port contract	Required
Modifying shared infrastructure (<code>scripts/</code> , <code>docs/agent-contract/</code> , etc.)	Required
Modifying <code>package.json</code> , root configs	Required
Adding a new module	Advisory --- claim the module name

2. Discovery Protocol

Before starting cross-boundary work, an agent executes this sequence:

1. Read `.claims/*.json` where `status == "active"` and `expires > now`
2. Compare own intended targets against active claims
3. If NO overlap -> file own claim, proceed
4. If overlap on EXTEND targets only -> file own claim with advisory note, proceed
5. If overlap on MODIFY/REPLACE targets -> STOP, resolve first

Integration with existing gates: The claim check should run as part of `pre-impl-gate.mjs`. A new script `scripts/claude/claim-check.mjs` handles the logic:

```
node scripts/claude/claim-check.mjs --targets modules/auth/public-api.mjs --action modify
```

Output:

```
claim-check: 1 active claim overlaps with your intended targets

CONFLICT: clm-a1b2c3 (team-alpha, TPL-099)
  target: modules/auth/public-api.mjs [modify]
  filed: 2026-04-03T10:00:00Z, expires: 2026-04-03T18:00:00Z
  strategy: modify-in-place

Resolution options:
  1. Wait for clm-a1b2c3 to complete
  2. Use BBA-additive strategy instead (add new export behind seam)
  3. File a negotiation claim (clm-a1b2c3-counter.json)
  4. Escalate to human
```

3. Conflict Resolution

Three tiers, escalating in cost:

Tier 1: Structural Avoidance (Zero Coordination Cost)

Convert the change from `modify` to `extend` :

Before (conflict-prone):

```
// Agent A wants to change auth port contract
// modules/auth/ports/auth-port.mjs
export function assertAuthPort(adapter) {
  // A wants to add refreshToken here --- but B is also modifying this
}
```

After (conflict-free via BBA):

```
// Agent A creates a NEW port version behind a seam
// modules/auth/ports/auth-port-v2.mjs (NEW FILE --- no conflict)
export function assertAuthPortV2(adapter) {
  assertAuthPort(adapter); // extends v1
  if (typeof adapter.refreshToken !== 'function') throw new Error('...');
}

// Agent A adds to public-api.mjs (additive --- merges cleanly)
export { assertAuthPortV2 } from './ports/auth-port-v2.mjs';
```

This is the preferred resolution. The old port stays untouched. Agent B's work is unaffected. The seam gets registered via `feature-seams` .

Tier 2: Sequencing (Low Coordination Cost)

When BBA isn't possible (e.g., fixing a bug in shared code), use `dependsOn` :

```
{
  "id": "c1m-d4e5f6",
  "agent": "team-beta",
  "slice": "TPL-124",
  "status": "active",
  "targets": [
    { "path": "modules/auth/ports/auth-port.mjs", "action": "modify", "surface": "port" }
  ],
  "strategy": "modify-in-place",
  "dependsOn": ["c1m-a1b2c3"],
  "notes": "Waiting for team-alpha to finish their auth-port changes first"
}
```

Agent beta files its claim but waits. When `c1m-a1b2c3` is completed, beta pulls the latest trunk and proceeds.

Tier 3: Human Escalation (Highest Cost)

When two agents need to modify the same surface simultaneously and neither can wait:

```
{
  "id": "clm-g7h8i9",
  "agent": "team-beta",
  "status": "active",
  "strategy": "negotiate",
  "targets": [...],
  "notes": "ESCALATION: Both team-alpha and team-beta need auth-port.mjs. Human decision needed."
}
```

The claim-check script flags `negotiate` claims prominently. A human reviews and decides ordering.

4. Signal Propagation

When Agent A completes a cross-boundary change, other agents need to know. Three mechanisms, from lightest to heaviest:

4a. Claim Status Update (Passive)

Agent A sets `"status": "completed"` in its claim file. Other agents polling `.claims/` see the resolution.

4b. Header Annotation (Ambient)

The file header's `CHANGELOG-BEGIN` block already records recent changes. An agent completing a cross-boundary change should update the target file's header changelog. Other agents reading headers will see:

```
CHANGELOG-BEGIN
Summary:
- Added OAuth2 PKCE refreshToken export behind bba-seam (claim clm-a1b2c3).
```

4c. Manifest Bump (Structural)

If a module's public API surface changed, bump the manifest version or add a note. Agents loading Tier 2 metadata (manifest.json) will notice the change.

5. BBA Interaction Details

Claims compose with BBA seams as follows:

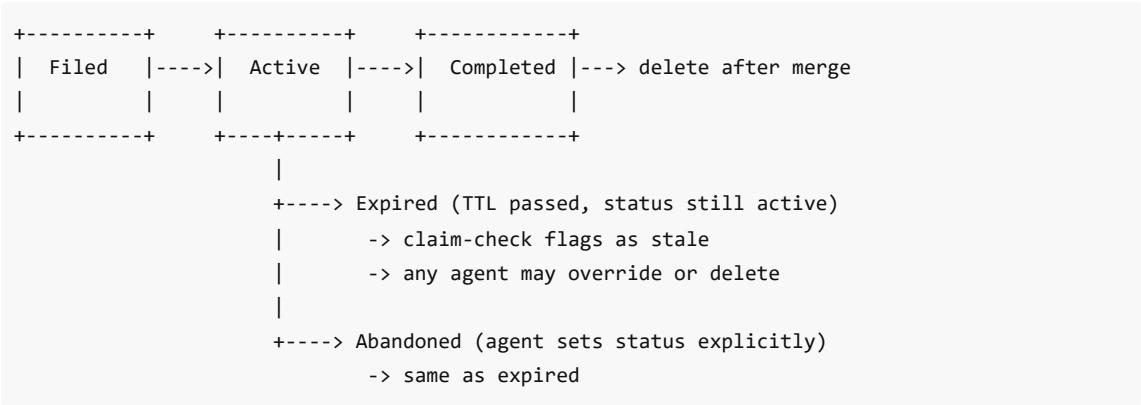
Claim Strategy	BBA Interaction
bba-additive	Agent creates a NEW seam in the target module. Registers it via <code>feature-seams</code> . Old path untouched.
modify-in-place	Agent modifies the ACTIVE path. Must coordinate via claim to prevent collision.
negotiate	Escalation --- BBA may be proposed as resolution by the human/reviewer.

Rule: When an agent files a `bba-additive` claim on another module, it creates a BBA seam there following the existing parallel-actor rule from ADR 0002:

When someone creates a temporary seam, they must document scope, owner, active path, and cleanup trigger in the nearest canonical place.

The claim file serves as the coordination artifact. The seam metadata (in headers/README) serves as the durable documentation.

6. Lifecycle and Cleanup



TTL defaults: 8 hours for agent sessions, 24 hours for human claims. Configurable per claim.

Orphan detection: `claim-check.mjs --audit` reports all claims where `expires < now && status == "active"` . These are presumed abandoned.

Cleanup: A pre-commit hook or periodic script can remove completed/expired claims older than 48 hours. Or they can be `.gitignore` 'd and treated as transient.

7. Comparison with Existing Systems

System	Mechanism	COA Analog
Database advisory locks	<code>SELECT ... FOR UPDATE NOWAIT</code>	Claim file + claim-check = advisory lock on file path
Distributed leases (etcd/ZK)	TTL-based lease with heartbeat	Claim with <code>expires</code> field, no heartbeat (stateless)
OS advisory locks (<code>flock</code>)	Cooperative, kernel-tracked	Cooperative, git-tracked
Google Piper (monorepo)	Change-level ownership + OWNERS files	Claims + module stewardship (header <code>steward</code> field)
Git merge	Three-way diff, last-writer-wins	Claims exist to prevent needing this, not to replace it
Kubernetes resource locks	Optimistic concurrency (<code>resourceVersion</code>)	First-commit-wins on trunk = optimistic concurrency

Key difference from all of the above: COA's primary mechanism is **structural avoidance** (BBA makes most changes additive), not locking. The claim system handles the residual cases where structural avoidance fails.

8. Minimal Viable Protocol (Phase 1)

The smallest mechanism that solves 80% of the problem:

What ships:

1. **Convention:** `.claims/` directory with a `README.md` explaining the format
2. **Rule:** "BBA-additive is the default strategy for cross-boundary changes" (add to ADR 0002 or create ADR 0008)
3. **Script:** `scripts/claude/claim-check.mjs` --- reads `.claims/*.json`, detects overlaps, reports advisory warnings
4. **Integration:** `pre-impl-gate.mjs` calls `claim-check.mjs` when the changed file set includes cross-module paths
5. **Agent instruction:** Add to `.claude/CLAUDE.md` agent routing: "Before modifying files outside your target module, check `.claims/` and file a claim"

What does NOT ship in Phase 1:

- No hard enforcement (advisory only)
- No negotiation files
- No automatic TTL enforcement
- No pre-commit hook integration
- No manifest/header annotations for active claims

Phase 1 user stories:

Happy path (no conflict):

```
Agent A receives slice TPL-123: "Add PKCE refresh to auth module"
1. Agent A reads .claims/ -> empty
2. Agent A files .claims/clm-a1b2c3.json (bba-additive, target: auth/public-api.mjs)
3. Agent A adds new export behind seam, commits
4. Agent A sets claim status to "completed" or deletes the file
```

Unhappy path (conflict detected):

```
Agent B receives slice TPL-124: "Fix auth port validation bug"
1. Agent B reads .claims/ -> sees clm-a1b2c3 targeting auth/public-api.mjs [extend]
2. Agent B's target is auth/ports/auth-port.mjs [modify] --- different file
3. claim-check reports advisory: "Nearby claim exists in auth module"
4. Agent B proceeds (different files, low risk)

--- OR ---

Agent B's target is ALSO auth/public-api.mjs [modify]
1. claim-check reports CONFLICT
2. Agent B checks: can this be reframed as bba-additive?
   - Yes -> reframe and proceed
   - No -> wait for clm-a1b2c3 to complete, then proceed
```

Degraded path (protocol ignored):

```
Agent C doesn't check .claims/
1. Agent C modifies auth/public-api.mjs directly
2. If Agent A already committed -> Agent C's commit may have a merge conflict
   -> git handles it
3. If Agent A hasn't committed -> both commit -> git merge may succeed
   (if changes are in different lines) or conflict (if same lines)
4. Damage is bounded: only the file they both touched is affected,
   hex boundaries protect everything else
```

9. Full Protocol (Phase 2+)

Building on Phase 1:

Phase	Addition
Phase 2	Pre-commit hook validates no active <code>modify</code> claims overlap with staged files. Claim lifecycle enforced (stale claims auto-flagged).
Phase 3	Negotiation protocol (counter-claims, priority ordering). Header/sidecar <code>ActiveClaim</code> metadata.
Phase 4	Dependency-aware claim ordering. Cross-repo claim federation (for multi-repo setups).

Each phase is independently valuable. Phase 1 alone handles most real-world cases because the BBA-first rule eliminates most conflicts structurally.

10. Failure Modes

Failure	Impact	Mitigation
Agent ignores protocol entirely	Possible merge conflict on shared files	Hex boundaries limit blast radius. Git merge is the backstop.
Orphaned claim (agent crashes)	Other agents may wait unnecessarily	TTL expiration. <code>claim-check --audit</code> flags stale claims.
Two agents file claims simultaneously	Both think they're clear	First-commit-wins on trunk. Loser rebases and retries.
Claim covers too many files	Over-broad lock, blocks others unnecessarily	Convention: claim only the files you'll actually modify. Scripts can warn on claims with >5 targets.
<code>.claims/</code> directory conflicts in git	Merge conflicts on claim files themselves	Claim files are additive (unique filenames). Completed/expired claims are deleted. Minimal conflict surface.

11. ADR-Ready Summary

Title: ADR 0008 --- Inter-Agent Coordination Protocol

Status: Proposed

Decision: Adopt a file-based claims protocol in `.claims/` for coordinating cross-boundary changes between parallel agents. The primary mechanism is a **BBA-first rule** (frame cross-boundary changes as additive BBA seams, not modifications). The claims mechanism handles residual cases where modification is unavoidable. Claims are **advisory** in Phase 1, with optional enforcement in later phases.

Consequences:

- (+) Parallel agents can work safely on the same repo
 - (+) No external services, databases, or central orchestrator needed
 - (+) Degrades gracefully --- ignoring the protocol doesn't break anything that hex + BBA doesn't already protect
 - (+) Composable with all existing COA primitives
 - (-) Adds one more thing agents must check before cross-boundary work
 - (-) Claim files add minor repo noise (mitigated by cleanup)
 - (-) Advisory-only enforcement means discipline is required
-

Design Constraints Satisfied

1. Works with trunk-based development (no long-lived feature branches) --- YES
2. File-based and repo-local (no external services) --- YES
3. Readable by agents with limited context windows (structured, compact) --- YES, claim files are ~15 lines JSON
4. Degrades gracefully (if an agent ignores the protocol, damage is bounded) --- YES, hex + BBA are the backstop
5. No central orchestrator or real-time communication required --- YES, git is the bus
6. Composes with existing COA primitives (headers, manifests, public-api, BBA seams) --- YES
7. Handles both agent-agent and agent-human coordination --- YES, claims are actor-type agnostic
8. Minimal ceremony for the common case (no cross-boundary work needed) --- YES, zero overhead when staying within module boundaries